

Professur für Rechnerarchitektur
Institut für Informatik
Technische Fakultät

Test und Zuverlässigkeit

Test and Reliability

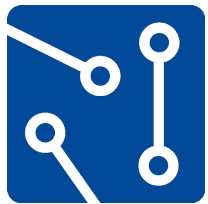
Summer Term 2026

Prof. Dr. Ralf Wimmer

Dr. Tobias Seufert

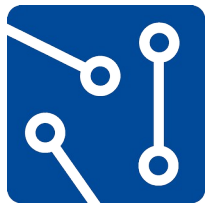
UNI
FREIBURG

Chapter 2: Fault Simulation



Fault Simulation

- Given a test set T and a fault list F , **find the faults** in F that are **detected** by T .
- Usage:
 - After test generation to find the set of faults detected (distinguished) by the test set. Yields the **fault coverage** of the test set.
 - During test generation to find the set of faults detected by each test.



Straight-forward Approach

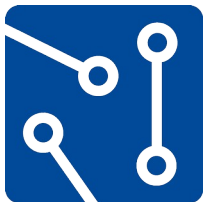
- Simulate T in the fault free circuit N .
- Change N into N_f that contains the fault f .
- Simulate T in N_f .
- Compare the output values.
- Repeat for every fault f in F .

Why is this a Challenge?

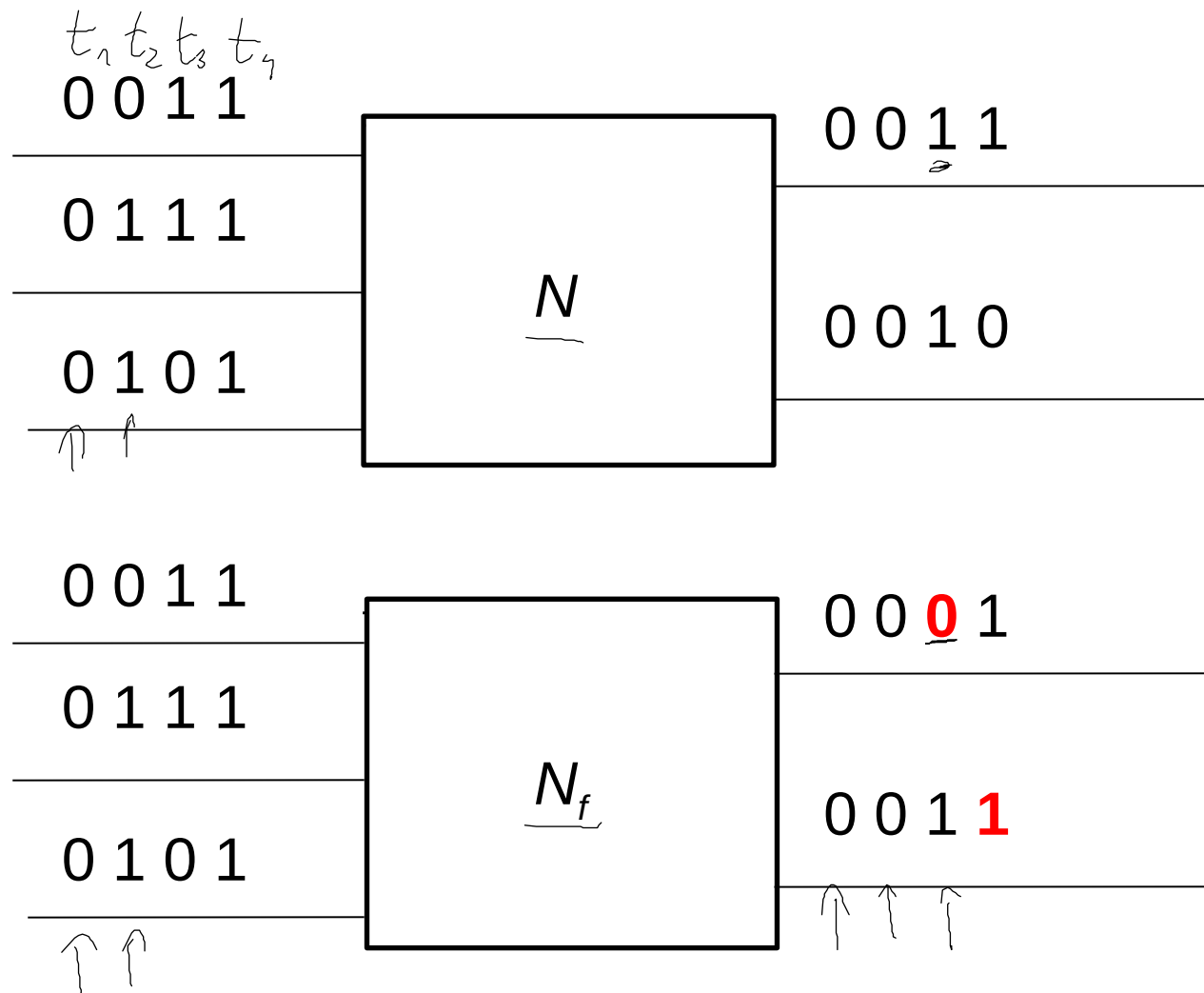
Circuit with 100 000 gates → at least 200 000 stuck-at-faults
(maybe a few less after EFC).

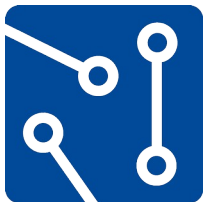
Assume 10 000 test patterns

→ **Naive simulation needs $200\,000 * 10\,000 * 100\,000$ steps!**

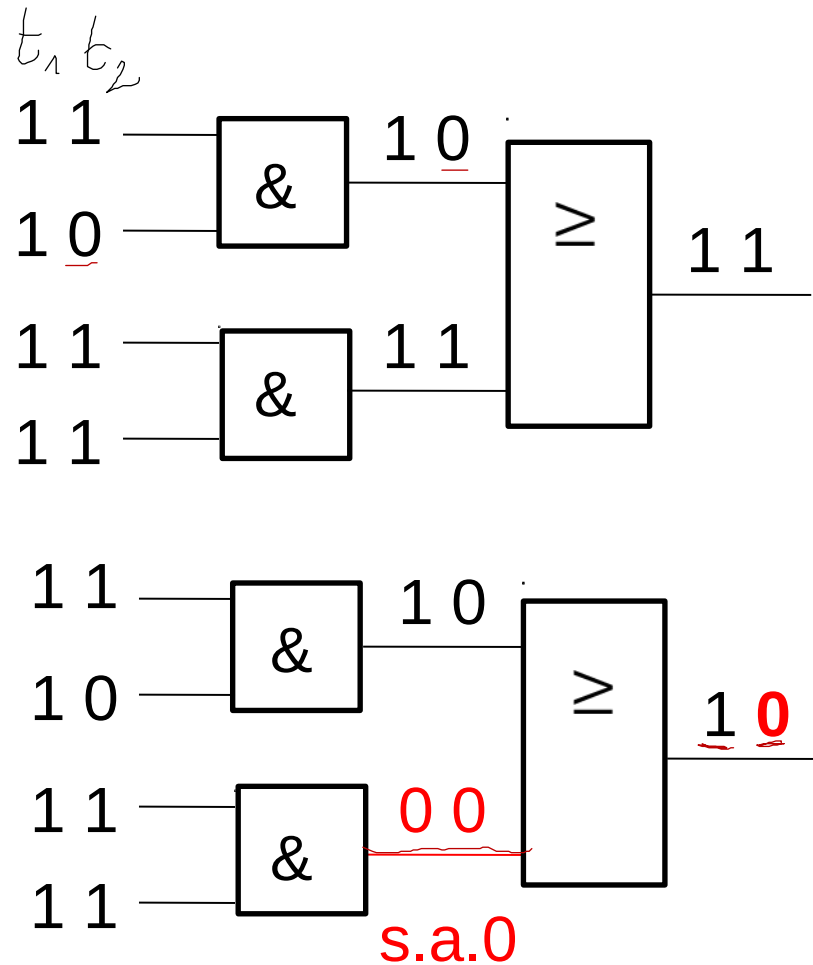


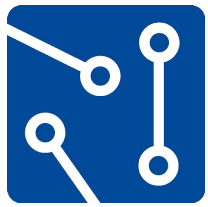
Example of the Simple Process





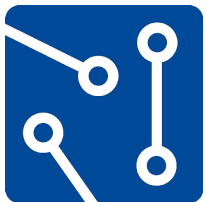
Specific Circuit





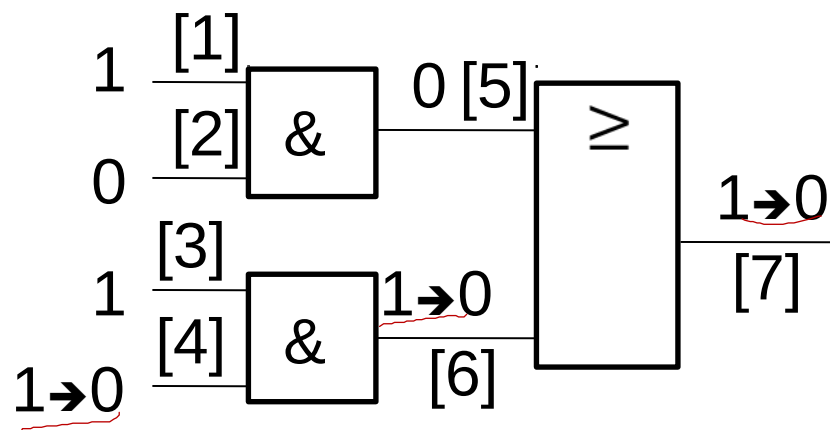
Efficient Simulation

- Event-driven simulation:
 - Take advantage of the fact that vectors are simulated one after the other.
 - Compute the value of a line only if the value of a line driving it **has changed**.

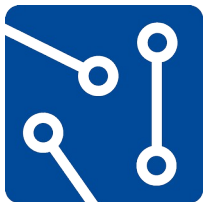


Example 1

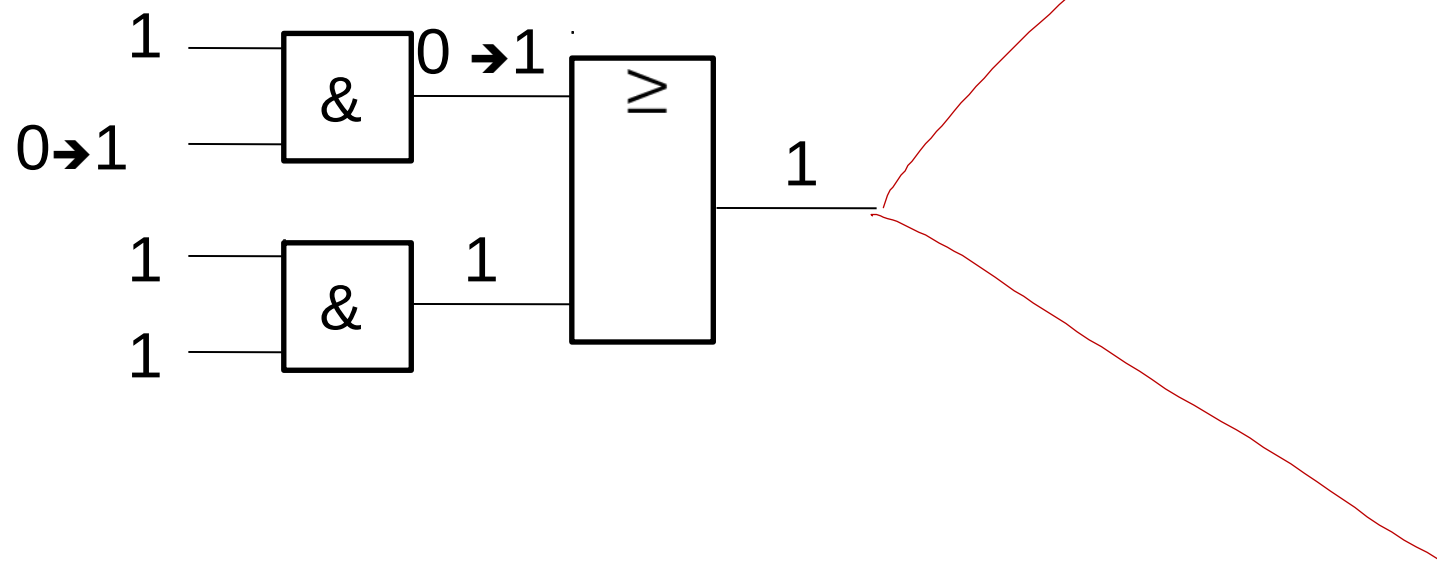
- Simulate 1011 followed by 1010

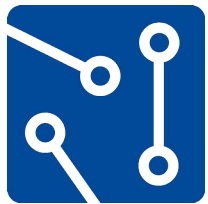


Events: <4> <6> <7>



Example 2





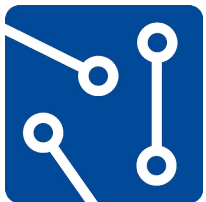
Fault Dropping

fault list

$$F = \{f_1, \dots, f_m\}$$

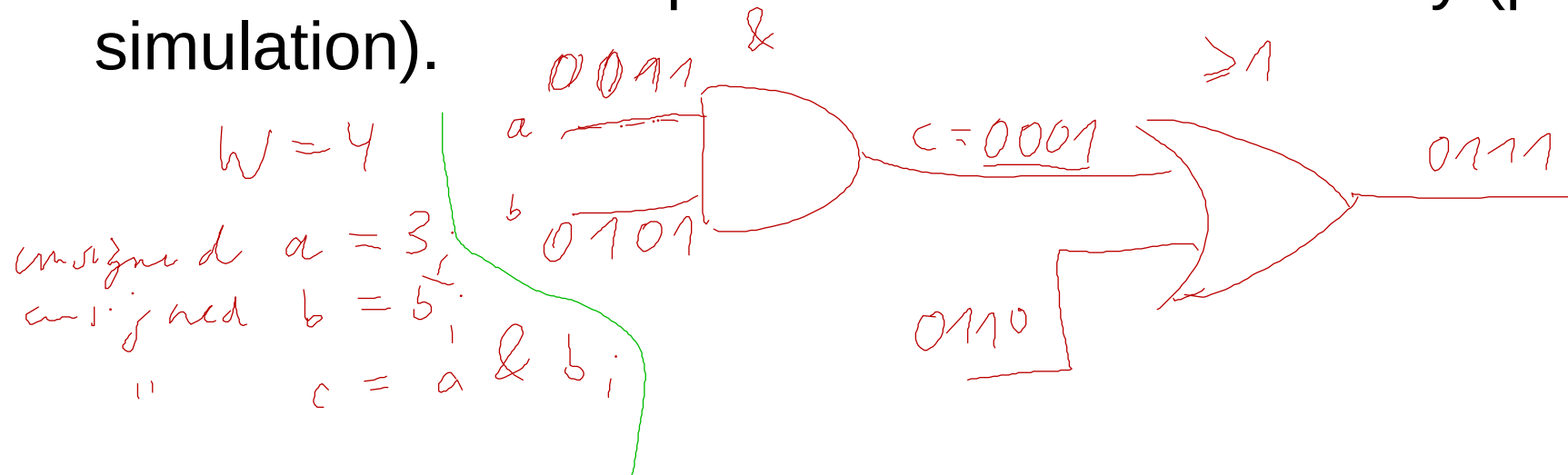
- 1 ■ To simulate f under $T = (t_1, t_2, \dots, t_n)$:
 - Simulate f under t_1 . If it is detected, stop.
 - Simulate f under t_2 . If it is detected, stop.
 - ...
- 2 ■ To simulate $F = \{f_1, f_2, \dots, f_m\}$ under T :
 - Simulate all the faults in F under t_1 .
 - Remove from F all the faults detected by t_1 .
 - Simulate all the faults in F under t_2 .
 - Remove from F all the faults detected by t_2 .
 - ...

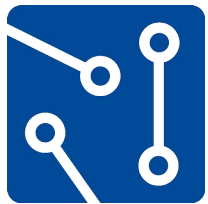
Fault dropping typically
not done when doing
fault **diagnosis**!



Parallel Solutions

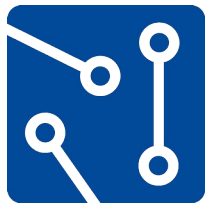
- A computer with word size W allows us to do operations on W bits **simultaneously**, typically $W = 64$ or more.
- Simulate the fault free and several faulty circuits simultaneously (parallel fault simulation).
- Simulate several input vectors simultaneously (parallel pattern simulation).





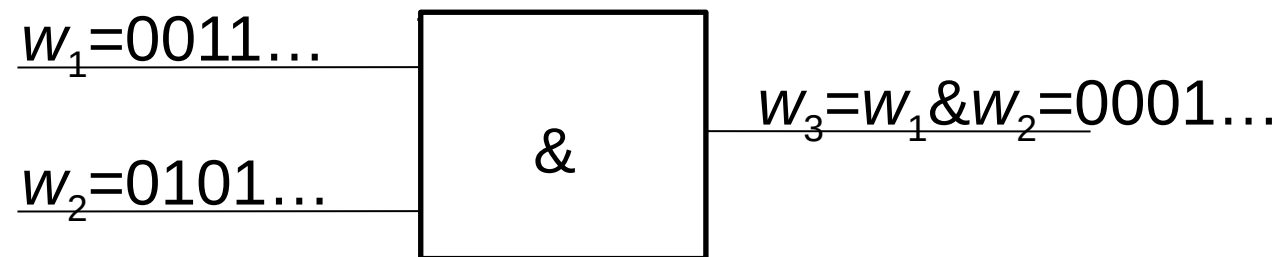
Parallel Fault Simulation

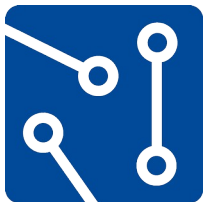
- Simulate one input vector at a time.
- One good circuit and $(W - 1)$ faulty circuits are **simulated simultaneously**.
- For a given line, the values of the **good circuit** and $(W - 1)$ **faulty circuits** are packed together into the same computer word of size W .
- Word operations are used to inject fault effects and compute output values.



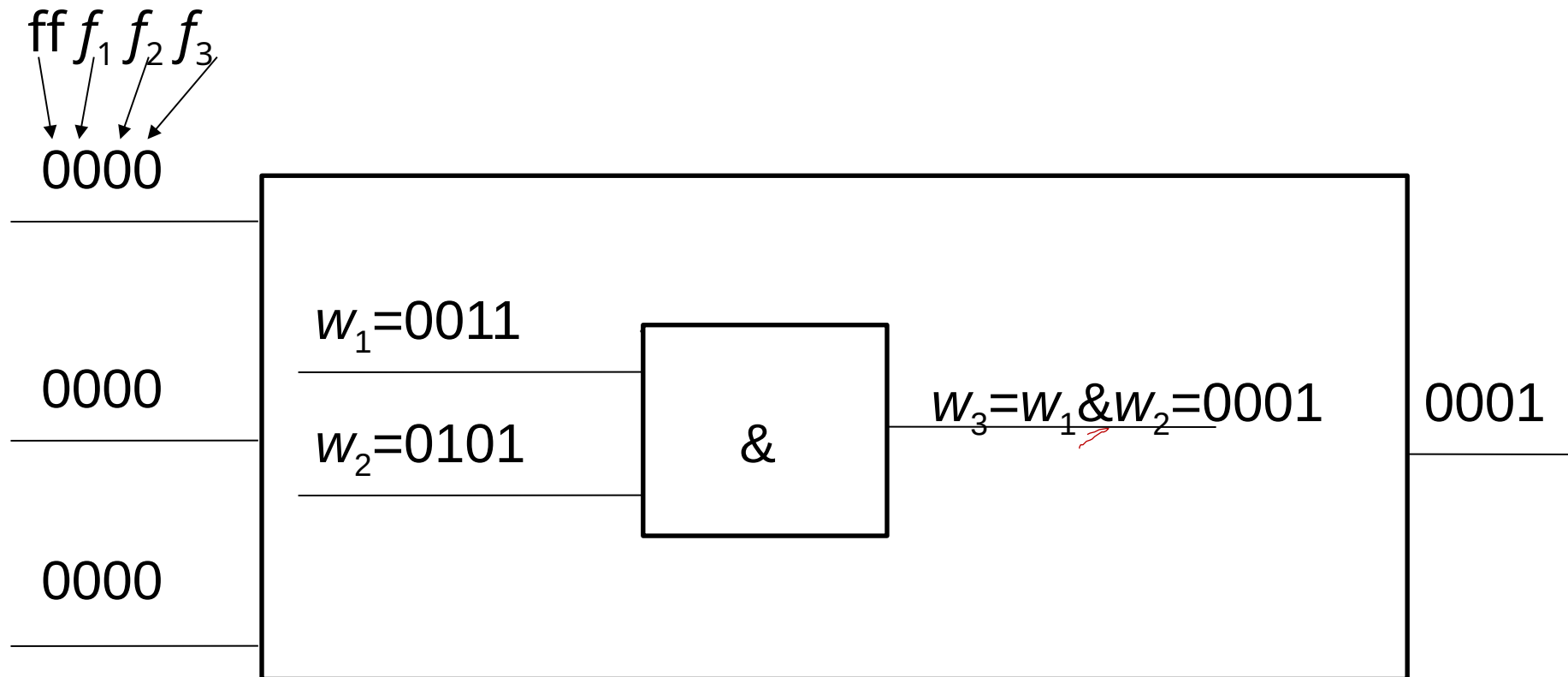
Example

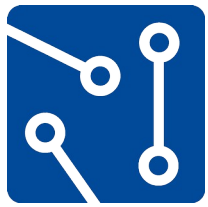
- Stuck-at faults in a combinational circuit require $\{0,1\}$ values.
- Assuming a computer word of 64 bits, 63 faults can be simulated in parallel (plus the fault-free circuit).
- A 2-input AND gate requires an AND operation on 2 words.





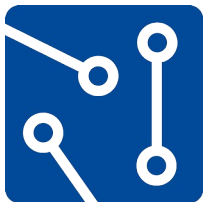
The Complete Circuit Test i (000)





Overall Procedure

- For a test $t = a_1a_2\dots a_n$, set the value of input j to $a_ja_j\dots a_j$.
- Compute line values from inputs to outputs using computer word operations appropriate for the gates; copy values of stems to their branches.
- Inject faults when necessary.



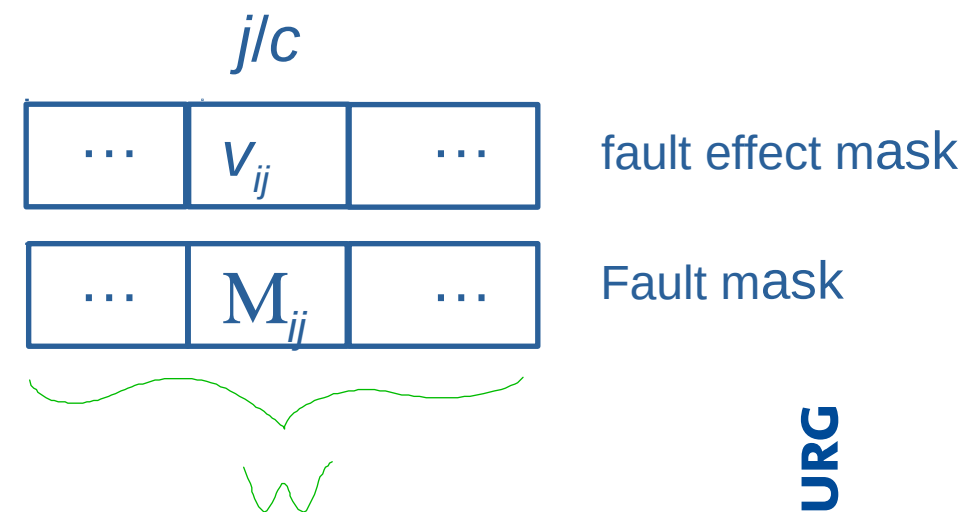
Faulty Lines

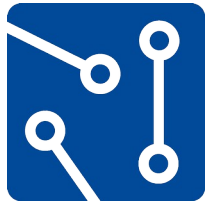
- Let v_{ij} be the value of line i in the faulty circuit N_f , where f is the fault j stuck-at c . v_{ij} is obtained by operations on gates driving line i .
- If $i = j$, the value of the line must be changed to c .
- The value of line i can be expressed as

$$V_{ij\text{ new}} := v_{ij} \cdot M_{ij}' + M_{ij} \cdot c$$

Line i

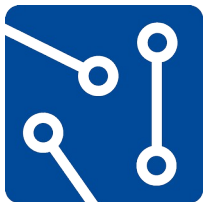
(where $M_{ij} = 0$ if $i \neq j$ and $M_{ij} = 1$ if $i = j$).



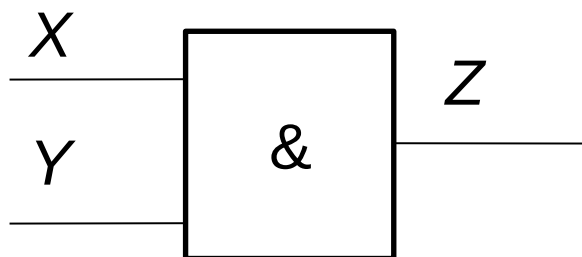


Fault Insertion for W Faults

- A line i has two masks:
 - I_i indicates the bit positions in which faults of line i should be inserted.
 - S_i indicates the stuck-at values of these faults.
- $$v_i = v_i \cdot I_i' + I_i \cdot S_i$$



Example



w

<u>ff</u>	X/1	Y/1	Z/0	Z/1	...
-----------	-----	-----	-----	-----	-----

"there is a fault"

I_Z

0	0	0	1	1	0
---	---	---	---	---	---

S_Z

			0	1	
--	--	--	---	---	--

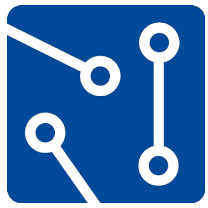
"the fault has this effect"

Z

z_1	z_2	z_3	z_4	z_5	z_6
-------	-------	-------	-------	-------	-------

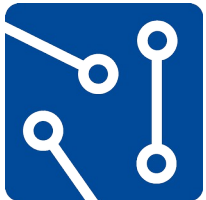
$$Z^* = Z \cdot I_Z' + I_Z \cdot S_Z$$

z_1	z_2	z_3	0	1	z_6
-------	-------	-------	---	---	-------



Advantages and Limitations of Parallel Simulation

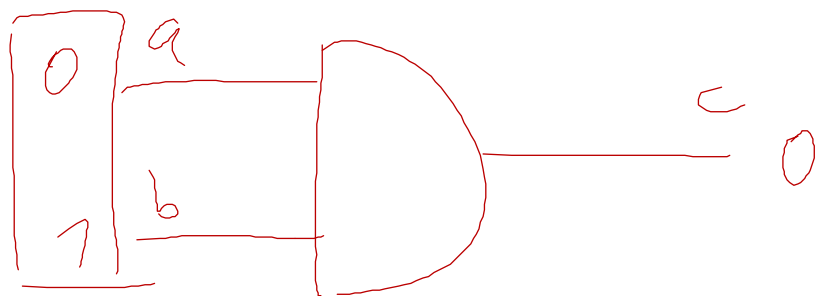
- + Divides fault simulation time by approximately W .
- Cannot take full advantage of event-driven simulation. A change of value in one circuit requires that all W circuits would be evaluated.
- Requires packing of values into words.
- Requires unpacking of values in order to access individual bits.
- Pattern-parallel simulation can only be applied if multiple patterns need to be simulated.



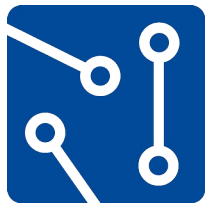
Deductive Fault Simulation

- Recovers the ability to use a form of event-driven simulation and potentially reduces storage.
- Considers **one test pattern** at a time.
- Starts with **fault-free** simulation of the test being considered.
- Considers all the faults simultaneously under the given test.
- Uses list operations on fault lists.

$$L_a = \{a/1\}$$
$$L_b = \{b/0\}$$

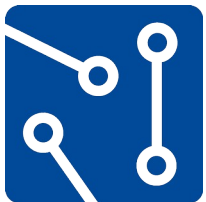


$$L_c = \{c/1\} \cup (L_a \setminus L_b)$$
$$= \{c/1, a/1\}$$

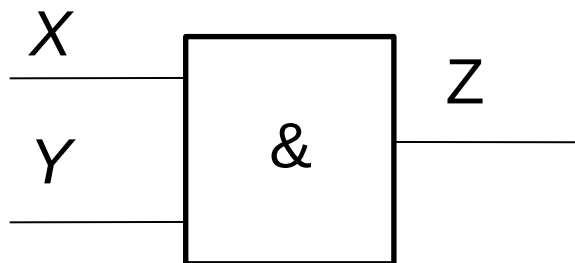


Fault Lists

- For every line i , a deductive fault simulator maintains a **list of faults L_i** .
- L_i is the set of faults f that cause the values of i in N and N_f to be different.
- For a primary input i with correct value a , $L_i = \{i / a\}$. / a
- For a primary output i , L_i is the set of faults detected at i assuming that all the values are $\{0,1\}$.



Relation to Parallel Fault Simulation



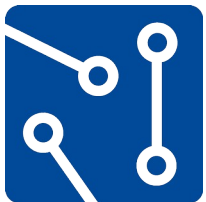
parallel

ff	X/1	Y/1	Z/0	Z/1	...
0	1	0	0	1	...

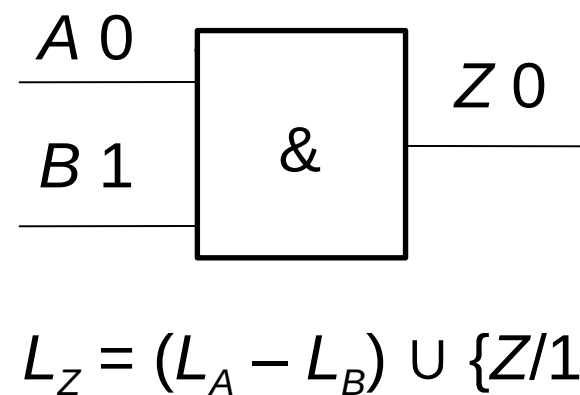
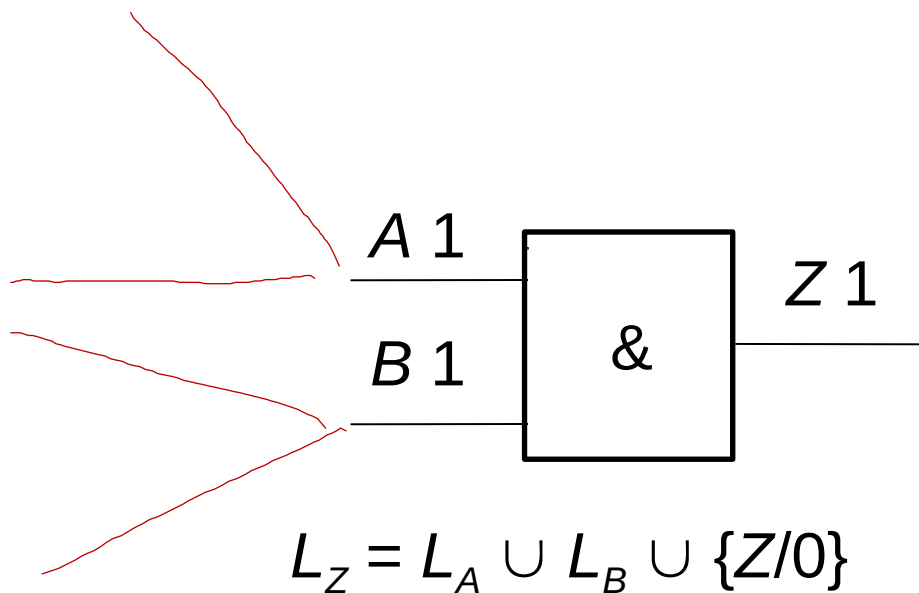
deductive

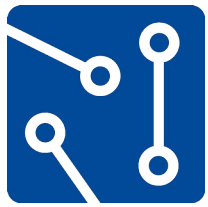
$$L_z = \{X/1, Z/1\}$$

Potentially reduces
storage requirements
and processing time by
storing only active
faults

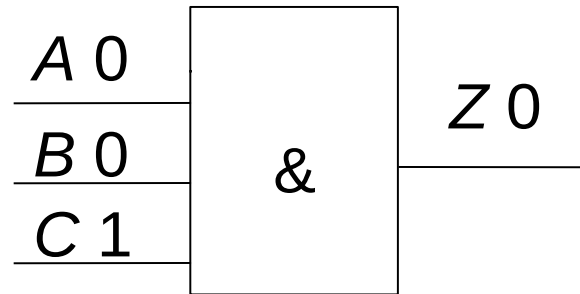


Fault List Propagation $\{0,1\}$ Values

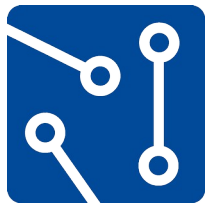




3-Input AND Gate

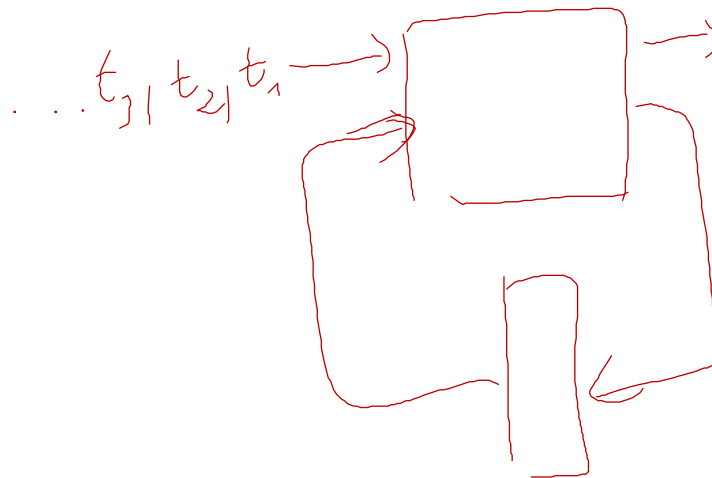


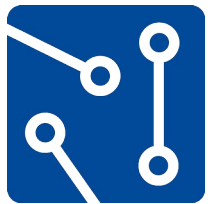
$$L_Z = ((L_A \cap L_B) - L_C) \cup \{Z/1\}$$



Sequential Circuits

- A test sequence $T = (t_1, t_2, \dots, t_n)$.
- To simulate t_{i+1} , the next-state under t_i must be known.
- t_{i+1} cannot be simulated in full before t_i is simulated.
- Nevertheless, parallel-pattern single-fault propagation is possible.





Fault Simulation Tools

- Academic (the first ones...)
 - Differential fault simulator [DAC-89]
 - PARIS [ICCAD-91]
- Commercial (integrated in ATPG)
 - Fastscan (Siemens EDA, formerly Mentor Graphics)
 - Tetramax (Synopsys)
 - Verifault (Cadence)
- More recently
 - Parallel beyond word-level parallelism (vector instructions etc.)
 - GPGPU-based

